Python Package Management with Anaconda and Conda

Environments, dependencies, channels, and reproducibility

T. Pranavan

ECE, NUS

Updated 2026

Why Package Management Matters

A common situation

- Project A needs Python 3.10 and NumPy 1.x
- Project B needs Python 3.12 and NumPy 2.x
- A system-wide upgrade fixes B but breaks A

The goal

Keep each project in a controlled, isolated software environment that can be rebuilt by another person.

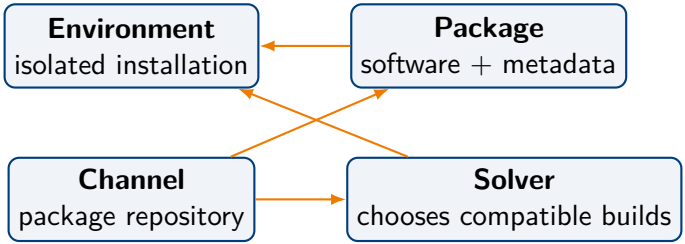
"It works on my machine" is not a reproducibility strategy.

Session Objectives

By the end of the session, you should be able to:

- distinguish Anaconda, Conda, Miniconda, Miniforge, and pip;
- create, activate, inspect, clone, and remove environments;
- install packages using versions and channels deliberately;
- combine Conda and pip with fewer dependency problems;
- export environments for portable or exact reproduction;
- diagnose common failures and recover from bad changes.

Four Concepts to Keep Separate



- Conda manages all four concepts.
- pip mainly installs Python packages from PyPI into the current Python environment.

Roadmap

What Is Anaconda? What Is Conda?

Conda A cross-platform package, dependency, and environment manager. It can manage Python itself and non-Python libraries.

Anaconda Distribution A large preconfigured distribution that includes Conda, Python, Jupyter tools, Navigator, and many packages.

Anaconda Navigator A graphical interface for launching tools and managing environments.

Anaconda repositories Package sources that Conda can use through channels such as `defaults`.

Important distinction

You can use **Conda without installing the full Anaconda Distribution**.

Choosing an Installer

Installer	What it contains	Good choice when	Trade-off
Anaconda	Conda plus a large collection of tools and packages	You want an all-in-one teaching or data-science setup	Large installation; many packages you may not use
Miniconda	Conda, Python, and a minimal base environment	You want a small official installer and will choose packages yourself	More initial setup
Miniforge	Minimal Conda-compatible installer with `conda-forge` as the default channel	You prefer a community-driven, lightweight setup	Requires understanding the chosen channel strategy

Practical recommendation

For project work, a minimal installer plus one environment per project is usually easier to maintain.

Installation and Verification

1. Install Anaconda, Miniconda, or Miniforge for your operating system.
2. Open the appropriate terminal or Anaconda Prompt.
3. Initialise shell integration when needed.

```
# Initialise the current shell (usually done once)
conda init

# Restart the terminal, then verify
conda --version
conda info

# Display help for any command
conda create --help
```

The Base Environment

What is it?

- The environment that contains the Conda installation itself
- Often activated automatically when a terminal starts
- Useful for maintaining Conda and basic command-line tools

Recommended habit

Do not use base as the environment for every project. Create a separate environment for each project or module.

```
# Optional: do not activate base automatically
conda config --set auto_activate_base false
```

Roadmap

Environment Lifecycle

Key mindset

An environment should be treated as **rebuildable**, not as a precious machine that must never be replaced.

Create, Activate, and Verify

```
# Create an environment and install core packages together
conda create -n vision python=3.12 numpy pandas matplotlib

# Activate it
conda activate vision

# Verify the active Python and environment
python --version
python -c "import sys; print(sys.executable)"
conda info --envs

# Leave the environment
conda deactivate
```

Why install core packages together?

The solver sees the full set of requirements at once and can choose a compatible combination.

Named Environments vs Project-Local Environments

Named environment

```
conda create -n mlproj python=3.12
conda activate mlproj
```

- Easy to activate by name
- Stored in Conda's environment directory

Environment at a path

```
conda create -p ./env python=3.12
conda activate ./env
```

- Clearly associated with a project
- Do not commit the environment directory to Git

Inspecting Environments and Packages

```
# List environments; the active one has an asterisk
conda env list
conda info --envs

# List packages in the active environment
conda list

# Show package sources (channels)
conda list --show-channel-urls

# Inspect a package in a named environment
conda list -n vision numpy
```

Clone, Rename, and Remove

```
# Clone before a risky upgrade or experiment
conda create -n vision-test --clone vision

# Rename an environment
conda rename -n vision-test vision-next

# Remove an entire environment
conda remove -n vision-next --all

# Verify
conda env list
```

Never delete environment folders manually

Use Conda commands so its environment registry remains consistent.

Run a Command Without Activating

```
# Useful in scripts, CI, and automated testing
conda run -n vision python train.py

# Check the interpreter used by an environment
conda run -n vision python -c "import sys; print(sys.executable)"
```

When this helps

`conda run` reduces reliance on shell activation in automated workflows.

Roadmap

Installing, Updating, and Removing Packages

```
conda activate vision

# Install packages
conda install scipy scikit-learn

# Install a constrained version
conda install "numpy>=2,<3"

# Update one package
conda update scikit-learn

# Remove a package
conda remove scipy
```

Read the transaction plan

Before confirming, check which packages will be installed, upgraded, downgraded, or removed.

Understanding Version Specifications

Specification	Meaning
`numpy`	Any compatible version
`numpy=2.1`	A compatible 2.1 release/build
`"numpy>=2,<3"`	At least 2.0, but below 3.0
`python=3.12`	Use Python 3.12 in this environment
`conda-forge::numpy`	Install NumPy specifically from conda-forge

Pin deliberately

Pin major/minor versions when compatibility matters, but avoid over-pinning every transitive dependency unless exact reproduction is required.

Search, Inspect, and Preview

```
# Search configured channels
conda search numpy

# Search one specific channel
conda search -c conda-forge pytorch

# Show detailed package records
conda search --info numpy

# Preview a transaction without changing the environment
conda install --dry-run numpy
```

A useful debugging question

Is the package unavailable, or is the requested combination of versions unsatisfiable?

What the Solver Does

- Builds a dependency graph from your requested packages.
- Applies version, platform, Python, and channel constraints.
- Selects compatible package builds.
- Proposes one transaction: installs, upgrades, downgrades, and removals.

Modern Conda

Current Conda releases use the `libmamba` solver by default, which substantially improves solving speed for many environments.

```
# Fallback for diagnosing unusual solver behaviour
conda install package-name --solver=classic
```

Conda Channels

- A channel is a repository containing Conda package builds and metadata.
- Common choices include `defaults` and `conda-forge`.
- Channel order affects which builds the solver considers first.
- Mixing channels carelessly can create incompatible binary stacks.

Choose a channel strategy

For a project, prefer one primary ecosystem and document it in `environment.yml`.

Channel Configuration and Strict Priority

```
# Show current configuration and channel order
conda config --show channels
conda config --show channel_priority

# Example: use conda-forge as the highest-priority channel
conda config --add channels conda-forge
conda config --set channel_priority strict

# Install from one channel without changing global config
conda install -c conda-forge seaborn
```

Strict priority

Once a package name is found in a higher-priority channel, lower-priority channel versions are excluded from consideration, reducing accidental mixing.

Roadmap

Conda vs pip

	Conda	pip
Primary role	Package, dependency, and environment management	Python package installation
Package source	Conda channels	Python Package Index (PyPI) and other Python indexes
Package scope	Python and non-Python libraries/tools	Python distributions
Python itself	Can install and switch Python versions	Uses an existing Python interpreter
Typical strength	Scientific stacks with compiled dependencies	Broad Python ecosystem and application packages

A Safer Conda + pip Workflow

```
# 1. Create an isolated environment that includes pip
conda create -n app python=3.12 pip
conda activate app

# 2. Install as much as possible with Conda
conda install numpy pandas matplotlib

# 3. Use pip only for remaining Python packages
python -m pip install package-not-available-in-conda

# 4. Check Python dependency consistency
python -m pip check
```

Rule of thumb

Conda first, pip last. If major Conda changes are needed after pip installation, rebuild the environment from a specification file.

Common Mistakes When Mixing Tools

- Running `pip` from a different Python installation than the active environment.
- Using `pip install --user`, which installs outside the environment.
- Alternating repeatedly between Conda and pip modifications.
- Assuming `pip freeze` fully captures non-Python libraries installed by Conda.
- Installing project packages into `base`.

Prefer this form

Use `python -m pip ...` so pip is tied to the active Python interpreter.

Roadmap

Anatomy of an environment.yml File

```
name: vision
channels:
  - conda-forge
  - nodefaults
dependencies:
  - python=3.12
  - numpy>=2
  - pandas
  - matplotlib
  - pip
  - pip:
      - albumentations==1.4.20
```

- The file documents environment name, channels, Conda dependencies, and optional pip dependencies.
- Commit the file to version control; do not commit the environment directory.

Create and Update from YAML

```
# Create a new environment
conda env create -f environment.yml

# Create it with a different name
conda env create -f environment.yml -n vision-dev

# Update an existing environment and remove unused packages
conda env update -f environment.yml --prune
```

Single source of truth

For team projects, edit the YAML file first, then update or rebuild the environment from it.

Export for Cross-Platform Sharing

```
# Export only packages explicitly requested by the user
conda export --from-history \
  --format=environment-yaml \
  --file=environment.yml
```

- Produces a smaller, clearer specification.
- Avoids many platform-specific transitive dependencies.
- The solver selects compatible builds on the recipient's machine.

Trade-off

Portable specifications improve cross-platform compatibility, but do not guarantee byte-for-byte identical environments.

Exact Reproduction on the Same Platform

```
# Export exact package URLs/builds
conda export --format=explicit --file=explicit.txt

# Recreate without solving again
conda create -n vision-copy --file explicit.txt
```

- Useful for deployment, archival, or exact testing.
- Usually tied to an operating system and CPU architecture.
- Less readable and less flexible than `environment.yml`.

Reproducibility Is a Spectrum

Approach	What is recorded	Portability	Reproduction
Manual commands	Human memory / notes	Poor	Poor
YAML from history	Direct requirements and channels	High	Similar compatible environment
Full YAML export	Direct and transitive packages	Medium	Closer, but platform-sensitive
Explicit lockfile	Exact package builds/URLs	Low	Exact on matching platform

Choose based on purpose

Collaboration usually prioritises portability; deployment and archival often prioritise exactness.

Environment History and Rollback

```
# Show environment revisions
conda list --revisions

# Restore a previous revision
conda install --revision 3
```

Rollback is helpful, but not a substitute for specifications

A committed environment file is easier to review, share, and rebuild than relying only on local history.

Roadmap

Using a Conda Environment in Jupyter

```
conda activate vision
conda install ipykernel

python -m ipykernel install --user \
  --name vision \
  --display-name "Python (vision)"

jupyter kernelspec list
```

- Select `Python (vision)` as the notebook kernel.
- The Jupyter server and the notebook kernel do not have to be in the same environment.

IDE Integration Checklist

- Select the interpreter inside the correct Conda environment.
- Confirm the terminal prompt shows the expected environment.
- Verify with `sys.executable` inside the IDE or notebook.
- Keep project dependencies in `environment.yml`.
- Exclude environment directories such as `env/` from Git.

Fast diagnostic

When an import works in the terminal but fails in the IDE, the IDE is usually using a different Python interpreter.

A Maintainable Project Layout

```
my-project/
|-- README.md
|-- environment.yml
|-- src/
|   `-- main.py
|-- notebooks/
|-- tests/
`-- .gitignore
```

README should include

- how to create the environment;
- how to activate it;
- how to run tests or the application;
- operating-system or hardware requirements;
- any required external data or services.

Roadmap

A Systematic Troubleshooting Sequence

1. Confirm the active environment and interpreter.
2. Inspect installed packages and channels.
3. Preview the requested change with `--dry-run`.
4. Simplify or relax incompatible version constraints.
5. Rebuild from a clean specification when necessary.

```
conda info --envs
python -c "import sys; print(sys.executable)"
conda list --show-channel-urls
conda install --dry-run package-name
```

Health Checks and Cache Cleanup

```
# Check the active environment for consistency problems
conda doctor

# Preview available automated repairs
conda doctor --fix --dry-run

# Remove unused caches and temporary files
conda clean --all --dry-run
conda clean --all
```

Use cleanup carefully

Review the dry run first, especially on shared systems or installations that use package-cache links.

Common Problems and Responses

Problem	Likely cause	Response
Package not found	Package/version absent from configured channels	Check spelling, platform, version, and channel availability
Version conflict	Conflicting constraints or channel builds	Relax pins, install requirements together, or use a clean environment
Import fails after install	Wrong interpreter/kernel	Check the Python executable and selected IDE/Jupyter interpreter
Environment became unstable	Repeated upgrades or Conda/pip mixing	Rebuild from YAML; compare with a fresh environment
Conda uses too much disk	Cached packages and old environments	Remove unused environments; inspect `conda clean` options

Instructor Demo: Build a Small Data Environment

```
conda create -n demo python=3.12 numpy pandas matplotlib
conda activate demo

python -c "import numpy, pandas, matplotlib; \
print(numpy.__version__)"

conda export --from-history \
  --format=environment-yaml \
  --file=environment.yml

conda deactivate
conda remove -n demo --all
```

Teaching prompt

Ask students to predict which files and packages remain after the environment is removed.

Hands-On Exercise 1: Environment Basics

Task

1. Create an environment named `prac1` with Python 3.12.
2. Install NumPy and Matplotlib.
3. Print the Python executable path and NumPy version.
4. List packages and identify their channels.
5. Clone the environment as `prac1-backup`.
6. Remove both environments.

Success criterion

You can explain why removing an environment does not uninstall your system Python or another Conda environment.

Hands-On Exercise 2: Reproducibility

Task

1. Create a project environment and install three packages.
2. Export a cross-platform `environment.yml` using `--from-history`.
3. Recreate the environment under a different name.
4. Compare the two outputs of `conda list`.
5. Export an explicit file and explain why it is less portable.

Discussion question

Which export would you commit to a student project repository, and which would you retain for an exact deployment snapshot?

Quick Knowledge Check

1. Why should project dependencies not be installed in `base`?
2. What is the difference between an environment and a channel?
3. Why can strict channel priority reduce dependency problems?
4. When should pip be used in a Conda environment?
5. What is the difference between `--from-history` and an explicit export?

Recommended Workflow

1. Install a minimal Conda distribution or the full Anaconda Distribution as appropriate.
2. Create one environment per project.
3. Choose and document a consistent channel strategy.
4. Install core dependencies together with Conda.
5. Use pip only for remaining Python packages.
6. Commit a human-readable environment specification.
7. Rebuild rather than repeatedly repairing a heavily modified environment.

Command Cheat Sheet

```
conda create -n ENV python=3.12
conda activate ENV
conda deactivate
conda env list
conda list
conda install PKG
conda update PKG
conda remove PKG
```

```
conda remove -n ENV --all
conda create -n NEW --clone OLD
conda install --dry-run PKG
conda export --from-history \
  --file=environment.yml
conda env create -f environment.yml
conda doctor
```

Further Reading

- Conda documentation: `https://docs.conda.io/`
- Managing environments: `https://docs.conda.io/projects/conda/en/stable/user-guide/tasks/manage-environments.html`
- Conda command reference: `https://docs.conda.io/projects/conda/en/stable/commands/`
- Conda-forge introduction and Miniforge: `https://conda-forge.org/docs/user/introduction/`
- Anaconda documentation: `https://www.anaconda.com/docs/`

Note

Package-manager commands and defaults evolve. Check the documentation installed with your Conda version using `conda COMMAND -help`.

Questions?

```
conda -help
```